

LAB 6

You are required to develop a backend application for managing media items (e.g., movies or songs) and collecting user reviews, where users can register, log in, browse media, and submit ratings and feedback. The system will include JSON API endpoints and an EJS-rendered HTML view for ratings.

Data Structure:

Data Description

The backend system manages three main collections in the MongoDB database: **Users**, **Medias**, and **Ratings**. Each collection has a specific schema and relationship to support user authentication, media management, and review functionalities.

1. Users Collection

Stores information about registered users of the system.

Field	Type	Description
_id	ObjectId	Unique identifier automatically generated by MongoDB
username	String	The user's login name (must be unique)
password	String	Hashed password stored securely using bcrypt
createdAt	Date	The timestamp when the account was created

Example document:

```
{
  "_id": "673a1b90f8cba46a78a1a90b",
  "username": "john_doe",
  "password": "$2b$10$34fghkl...",
  "createdAt": "2025-10-25T09:00:00Z"
}
```

2. Medias Collection

Contains details about media items such as movies, music, or other digital content.

Field	Type	Description
_id	ObjectId	Unique identifier for each media item
title	String	Title of the media (movie name, song name, etc.)
category	String	Type or genre (e.g., Action, Drama, Pop, etc.)
year	Number	Release year
description	String	Short summary or details about the media
posterUrl	String	URL of the media's poster image
averageRating	Number	Average user rating, calculated dynamically from the Ratings collection

Example document:

```
{
  "_id": "673a1c22e9b4f45c4b23d8b2",
  "title": "Inception",
  "category": "Sci-Fi",
  "year": 2010,
  "description": "A skilled thief leads a team into dreams to steal secrets.",
  "posterUrl": "/images/inception.jpg",
  "averageRating": 4.7
}
```

3. Ratings Collection

Stores user ratings and comments for each media item.

Field	Type	Description
_id	ObjectId	Unique identifier for each rating
mediaId	ObjectId	Reference to a document in the Medias collection
userId	ObjectId	Reference to a document in the Users collection
rating	Number	User's rating score (1–5)
comment	String	Text feedback from the user
createdAt	Date	Date and time the rating was submitted

Example document:

```
{
  "_id": "673a1c73b91a3c56b91f8b2e",
  "mediaId": "673a1c22e9b4f45c4b23d8b2",
  "userId": "673a1b90f8cba46a78a1a90b",
  "rating": 5,
  "comment": "Mind-bending and visually stunning!",
  "createdAt": "2025-10-25T10:15:00Z"
}
```

Relationships:

- Each **User** can submit multiple **Ratings**.
- Each **Media** can have multiple **Ratings** from different users.
- Ratings link **Users** and **Medias** via `userId` and `mediaId`.

General Requirements:

- A **resource** folder is provided, containing sample JSON data, a list of required npm packages for reference and development, example API endpoints for testing using tools like Postman or Curl, and an EJS template file for rendering views.
- Create the project folder **Mediasys_Studentcode**.
- Development Tools: Visual Studio Code, Postman/Curl, MongoDB Compass, Web Browser, etc.
- Backend Technologies: Node.js, Express.js, EJS, MongoDB, Mongoose, Dotenv, JWT, CORS, etc.

Tasks to be Completed:

Task 1: Project Initialization and Configuration

In this task, you are required to set up the database structure and environment configuration for the Media Rating System:

- Create a database named **MyMediaDB_2025** and set up the required data structure for the application. Create three collections: **Users**, **Medias**, and **Ratings**, following the provided schema. Then, import at least a few sample documents into each collection (you may use the provided sample JSON files or create your own test data). See **Figure 1** for more details.

- Create a **.env** file to define essential environment variables, including the server port (**PORT**), MongoDB connection string (**MONGO_URI**), and JWT secret (**JWT_SECRET**). This helps your application load configuration securely and ensures the server runs on <http://localhost:1234>.

Task 2: API – GET /medias

- Create an API endpoint using the GET method at **/medias** that does not require authentication.
- This API must **fetch all media items** from the medias collection and return them in JSON format. Each Media item in the response must include all relevant information, as illustrated in **Figure 2**.

Task 3: API – POST /users/register

- Create an API endpoint that allows a new user to register by sending a username and password in the request body using the POST method. Upon successful registration, return a JSON message confirming that the user was created. If the username already exists or an error occurs, return an appropriate failure message.
- The server must hash the password using **bcrypt** before storing it in the Users collection to ensure security. See **Figure 3** for more details.

Task 4: API – POST /users/login

- Create an API endpoint using the POST method at **/users/login** that allows a user to log in with a username and password.
- This API must validate the credentials by comparing the password with the hashed version in the database, and return a signed **JWT token** in JSON format if authentication succeeds. If invalid, return an appropriate error message. See **Figure 4** for more details.

Task 5: API – GET /ratings/myreviews (Authenticated)

In this task, you will create a protected API endpoint that allows a logged-in user to view **all ratings they have submitted**, including detailed information about the media items they reviewed.

Requirements:

1. The endpoint must be accessible at:
2. GET /ratings/myreviews
3. Only authenticated users (with valid JWT tokens) can access this route.
4. Use **Mongoose populate()** to display related media information for each rating.
5. The returned JSON should include:
 - The rating score and comment
 - The media title, category, and release year
 - The date the rating was created

Example response:

```
{
  "user": "john_doe",
  "reviews": [
    {
      "media": {
        "title": "Inception",
        "category": "Sci-Fi",
        "year": 2010
      },
      "rating": 5,
      "comment": "Mind-bending and visually stunning!",
      "createdAt": "2025-10-25T10:15:00Z"
    },
    {
      "media": {
        "title": "Titanic",
```

```

        "category": "Romance",
        "year": 1997
    },
    "rating": 4,
    "comment": "Heartbreaking but beautiful story.",
    "createdAt": "2025-10-25T11:00:00Z"
  }
]
}

```

Technical notes:

- Require JWT authentication middleware to decode and verify the token.
- Use query:
- `Rating.find({ userId: req.user.id }).populate('mediaId', 'title category year')`
- Return data in JSON format.

Task 6: API – GET /medias/details/:id (Authenticated)

In this task, you will create a detailed media information endpoint that includes **all related user reviews**, using Mongoose's **nested populate()** feature.

Requirements:

1. The endpoint must be accessible at:
2. GET /medias/details/:id
3. Only authenticated users can access this endpoint.
4. For the specified media ID, the response must include:
 - Basic media information (title, description, category, year, poster URL, etc.)
 - A list of all ratings and reviewer usernames associated with the media.
5. Use **populate()** to retrieve both rating and user details in one query.

Example response:

```

{
  "_id": "673a1c22e9b4f45c4b23d8b2",
  "title": "Inception",
  "category": "Sci-Fi",
  "year": 2010,
  "description": "A skilled thief leads a team into dreams to steal secrets.",
  "posterUrl": "/images/inception.jpg",
  "averageRating": 4.7,
  "reviews": [
    {
      "rating": 5,
      "comment": "Amazing visuals and plot!",
      "user": {
        "username": "john_doe"
      },
      "createdAt": "2025-10-25T10:15:00Z"
    },
    {
      "rating": 4,
      "comment": "Complex but enjoyable.",
      "user": {
        "username": "alice_w"
      },
      "createdAt": "2025-10-25T11:30:00Z"
    }
  ]
}

```

Technical notes:

- Use nested population query:
- `Media.findById(req.params.id)`
- `.populate({`
- `path: 'ratings',`
- `populate: { path: 'userId', select: 'username' }`
- `});`
- The endpoint must verify JWT tokens before executing the query.
- Return JSON response for EJS or API usage.

Security Reminder

All API endpoints from **Task 5** and **Task 6** require JWT token verification in the request header, for example:

Authorization: Bearer <token>

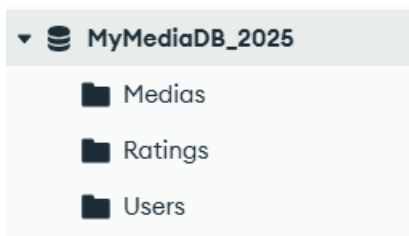


Figure 1 – Database structure

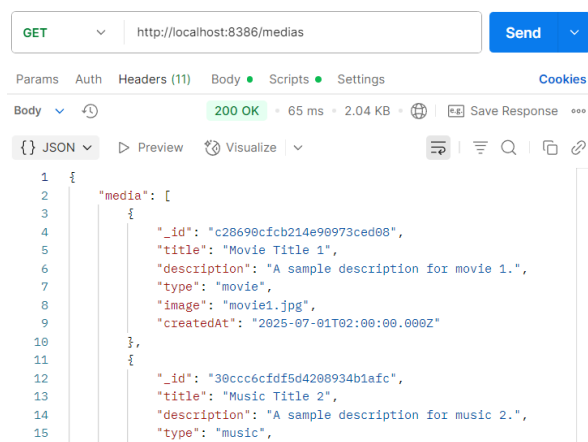


Figure 2 - JSON response from the `GET /medias` API, displaying a list of all media items <http://localhost:8386/medias>

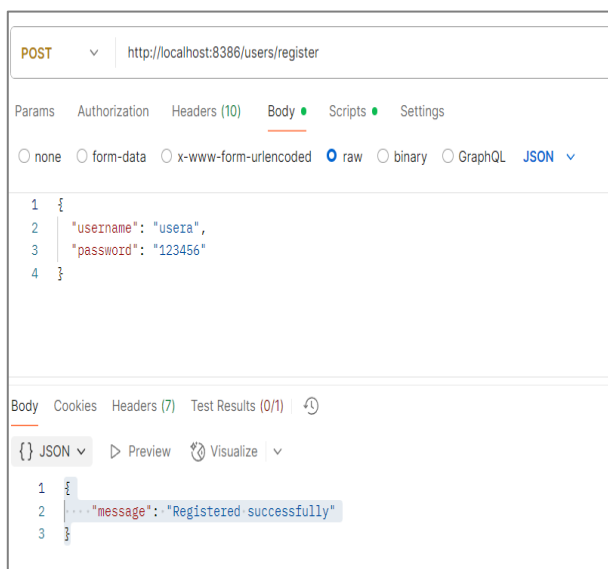


Figure 3 - Response after executing POST /users/register API to create a new user:
<http://localhost:8386/users/register>

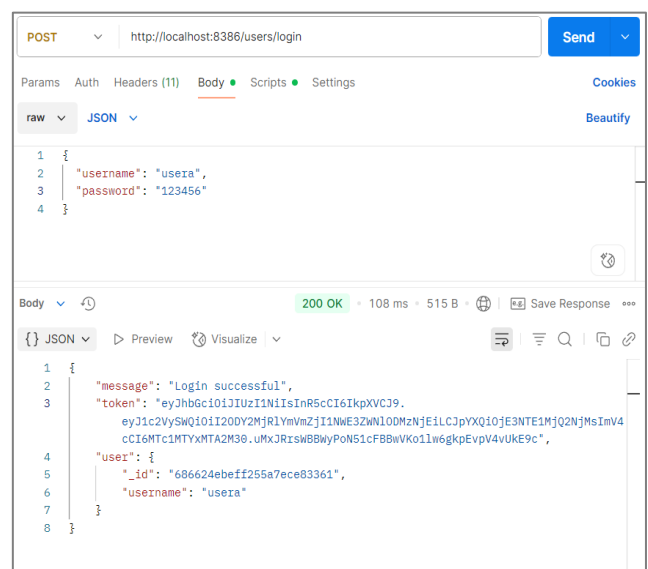


Figure 4 – *JWT token returned after successful login via POST /users/login API:*
<http://localhost:8386/users/login>

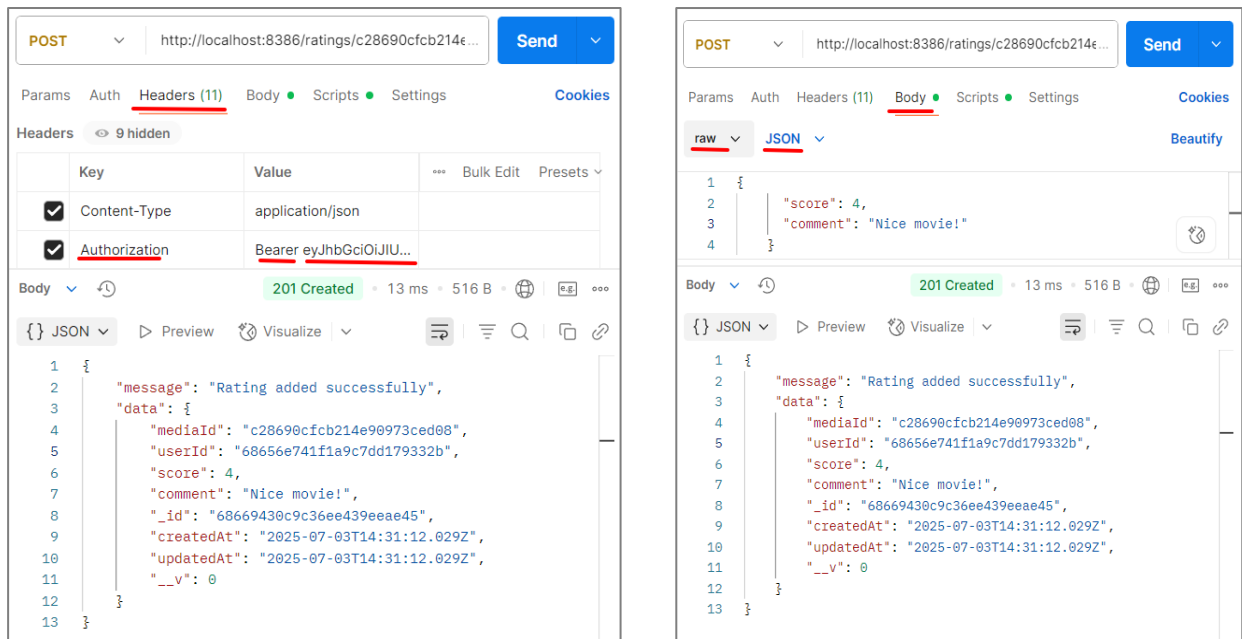


Figure 5 – Result after executing the submit rating API at:
<http://localhost:8386/ratings/c28690cfcb214e90973ced08>